

长广溪智能制造（无锡）有限公司

Changguangxi Intelligent Manufacturing (Wuxi) Co., Ltd.

Socket 通讯手册

文档版本：V1.2

发布日期：2025 年 10 月 27 日

文档编号：CGXi-QM-CR-0014



长广溪智能制造（无锡）有限公司

Changguangxi Intelligent Manufacturing(Wuxi) Co., Ltd.



长广溪智能制造（无锡）有限公司（简称长广溪智造）保留对此文件修改之权利且不另行通知。长广溪智能制造（无锡）有限公司所提供之信息相信为正确且可靠之信息，但并不保证本文件中绝无错误。请于向长广溪智能制造（无锡）有限公司提出订单前，自行确定所使用之相关技术文件及规格为最新之版本。若因贵公司使用本公司之文件或产品，而涉及第三人之专利或著作权等智能财产权之应用及配合时，则应由贵公司负责取得同意及授权，本公司仅单纯贩售产品，上述关于同意及授权，非属本公司应为保证之责任。又未经长广溪智能制造（无锡）有限公司之正式书面许可，本公司之所有产品不得使用于医疗器材，维持生命系统及飞航等相关设备。

目录

一 简介	2
二 机器人的设定与程序的编写	3
2.1 IP 地址设定	3
2.2 常用代码脚本	3
2.2.1 socket_connect	3
2.2.2 socket_disconnect	3
2.2.3 socket_open	3
2.2.4 socket_close	4
2.2.5 socket_receive	4
2.2.6 socket_send	5
2.2.7 socket_receive_string	5
2.2.8 socket_send_string	6
2.2.9 socket_read_string	6
2.2.10 get_socket_connect_status	7
2.3 程序的编写	8
2.3.1 机器人作为客户端	8
2.3.2 机器人作为服务器	9
三 通讯功能的实现	12
3.1 机器人作为客户端	12
3.2 机器人作为服务器	12

一 简介

CGXi 机器人与上位机或工业相机通过 TCP/IP 协议的通讯连接，实现机器人与上位机或工业相机之间数据传输的功能。

二 机器人的设定与程序的编写

2.1 IP 地址设定

机器人与上位机或工业相机需要在相同的网段下，常用如 192.168.6.***，机器人修改 IP 地址可查询 CGXi 用户手册。

2.2 常用代码脚本

设定完 IP 后，需要在编程界面编写脚本程序来定义具体通讯规格，下列为 socket 通经常用脚本代码。

2.2.1 socket_connect

描述

作为客户端连接服务器。

参数

const char *TCPorUDP: 协议类型 "TCP" (支持最多 16 个) 或者 "UDP"(支持最多 8 个);

const char *ip: 服务器 IP 地址;

int SocketID: socket ID, 取值范围 2-23;

int portNo: 端口号, 与服务器相同。

示例

```
socket_connect("tcp","192.168.6.201",8,9000)  --请求控制器创建一个客户端
```

2.2.2 socket_disconnect

描述

作为客户端断开连接。

参数

int SocketID: socket ID。

示例

```
socket_disconnect(8)  --断开 SocketID 为 8 的连接
```

2.2.3 socket_open

描述

作为服务器打开端口。

参数

const char *TCPorUDP: 协议类型 "TCP" (支持最多 16 个) 或者 "UDP"(支持最多 8 个);

int SocketID: socket ID, 取值范围 2-23;

int portNo: 端口号。

示例

```
socket_open("tcp",8,9000)           --请求控制器打开控制器端口
```

2.2.4 socket_close

描述

作为服务器关闭端口。

参数

int SocketID: socket ID。

示例

```
socket_close(8)                     --关闭 SocketID 为 8 的端口
```

2.2.5 socket_receive

描述

网口数据接收。接收数据并写入指定通讯类型变量的起始偏移地址处；接收完成后可调用 get_last_receive_status() 函数获取接收的结果。假如发送过来的十六进制数据为“12 34 56 78 AB CD”，则接收到的 16 位整数寄存器十六进制为“12 34 56 78 AB CD”。

参数

int socketID: socket ID;

int num: 通讯变量个数;

CommuVarType regType: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex: 通讯变量起始偏移量;

int timeout: 接收超时时间, 单位毫秒; 若 <= 0, 无限等待直到接收到数据。

返回值

无。

示例

```
socket_receive(6, 8, 1, 0, 2000)
```

--接收 8 个 int16 类型的数据，接收到后从 int16 通讯变量寄存器的偏移地址 0 开始依次存入。

2.2.6 socket_send

描述

网口数据发送。

参数

int socketID: socket ID;

int num: 通讯变量个数;

CommuVarType regType: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex: 通讯变量起始偏移位。

示例

```
socket_send(8,1000,1,0)           --发送指定通讯类型变量地址处的数据
```

2.2.7 socket_receive_string

描述

网口字符串接收。收字符串并写入指定类型的通讯变量地址处；接收完成后可调用 get_last_receive_status() 函数获取接收状态。假如发送过来的十六进制数据为“12 34 56 78 AB CD”，则接收到的 16 位整数寄存器十六进制数据为“34 12 78 56 CD AB”。

参数

int socketID: socket ID;

int num: 接收字符最大个数;

CommuVarType regType: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex: 通讯变量起始偏移量;

int timeout: 接收超时时间, 单位毫秒; 若 <= 0, 无限等待直到接收到数据。

返回值

无。

示例

```
socket_receive(6, 8, 1, 0, 2)

recv_status = get_last_receive_status()

if(recv_status == 1)
```

```
then
    str = commuvar_to_string(8,1,0)
    print(str)
else
    print("recv fail")
end
```

--接收 8 个字符的数据，接收到后从 int16 通讯变量寄存器的偏移地址 0 开始依次存入。

2.2.8 socket_send_string

描述

网口字符串发送。发送指定字符串并存放在指定通讯类型变量地址处。

参数

int socketID: socket ID;

CommuVarType regType: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex: 通讯变量起始偏移量;

const char *str: 发送字符串。

示例

```
socket_send_string(6,1,1,"(6,1.1,2.2,3.3,4.4,5.5,6.6)")
```

--socketID 为 6，通讯变量类型为 int16，通讯变量起始偏移量为 1，发送字符串为
“6,1.1,2.2,3.3,4.4,5.5,6.6”

2.2.9 socket_read_string

描述

读取指定端口的字符串。

参数

int socketID: socket ID;

int timeout 接收超时时间，单位毫秒；若<=0，无限等待直到接收到数据。

示例

```
str = socket_read_string(6,2000)
```

--从 socketID 为 6 的端口读取字符串并赋值给变量 str，接收超时时间为 2000ms

2.2.10 get_socket_connect_status

描述

获取 Socket Connect 的状态。

参数

int socketID

返回值

int status: connect status, 0-连接成功, 1-连接失败, 2-连接中, 3-非法连接。

示例

```
socket_connect("tcp", "192.168.6.100", 6, 8000)

timeoverridecount = 20

while(timeoverridecount)
do
    connect_status = get_socket_connect_status(6)
    if(connect_status == 0)
        then
            print("connect ok")
            break
        elseif(connect_status == 1)
            then
                print("connect fail")
                break
            elseif(connect_status == 3)
                then
                    print("connect invalid")
                    break
            end
        timeoverridecount = timeoverridecount - 1
    wait(500)
```

```
end
```

2.3 程序的编写

机器人作为客户端或服务端时，编程稍有不同，下面将分别作介绍。

2.3.1 机器人作为客户端

首先需要释放指定 ID 的控制器客户端,例如释放 socket ID 为 8 的客户端:

```
socket_disconnect(8)
```

然后请求控制器创建一个客户端，例如创建如下一个客户端：

```
socket_connect("tcp","192.168.6.201",8,9000)
```

--此客户端协议类型为“tcp”，服务器 IP 地址为“192.168.6.201”，socket ID 为 8，端口号为 9000

此时可添加一个读取 socket 连接状态的脚本（get_socket_connect_status）来判断是否通讯成功。

```
socket_connect("tcp", "192.168.6.100", 6, 8000)

timeoverridecount = 20

while(timeoverridecount)
do
    connect_status = get_socket_connect_status(6)

    if(connect_status == 0)
    then
        print("connect ok")    --通讯成功日志提示"connect ok"
        break
    elseif(connect_status == 1)
    then
        print("connect fail")  --通讯失败日志提示"connect fail"
        break
    elseif(connect_status == 3)
    then
        print("connect invalid") --非法通讯日志提示"connect invalid"
        break
    end
end
```

```

timeoverridecount = timeoverridecount - 1

wait(500)

end

```

最后编写接收与发送脚本，例如：

```

socket_send(8,1000,1,0)

--发送指定通讯类型变量通讯地址的数据

--socket ID 为 8

--通讯变量个数为 1000

--通讯变量类型为 int16

--通讯变量起始偏移量为 0

```

```

socket_receive(8,1000,1,0,1000)

--接受数据并写入指定通讯类型变量通讯地址的数据

--socket ID 为 8

--通讯变量个数为 1000

--通讯变量类型为 int16

--通讯变量起始偏移量为 0

--超时时间 1000ms

```

2.3.2 机器人作为服务器

首先需要关闭指定 ID 的控制器服务器,例如释放 socket ID 为 6 的服务器：

```

socket_close(6)

```

然后请求控制器创建一个服务器，例如创建如下一个服务器：

```

socket_open("tcp",6,8080)

--此服务器端协议类型为“tcp”，socket ID 为 6，端口号为 8080

```

此时可添加一个读取 socket 连接状态的脚本（get_socket_connect_status）来判断是否通讯成功。

```

socket_connect("tcp", "192.168.6.100", 6, 8000)

timeoverridecount = 20

while(timeoverridecount)

```

```
do

    connect_status = get_socket_connect_status(6)

    if(connect_status == 0)

        then

            print("connect ok")      --通讯成功日志提示"connect ok"

            break

        elseif(connect_status == 1)

            then

                print("connect fail")  --通讯失败日志提示"connect fail"

                break

            elseif(connect_status == 3)

                then

                    print("connect invalid") --非法通讯日志提示"connect invalid"

                    break

                end

            timeoverridecount = timeoverridecount - 1

            wait(500)

        end

    end
```

最后编写接收与发送脚本，例如：

```
socket_send(6,1000,1,0)

--发送指定通讯类型变量通讯地址的数据

--socket ID 为 6

--通讯变量个数为 1000

--通讯变量类型为 int16

--通讯变量起始偏移量为 0
```

```
socket_receive(6,1000,1,0,1000)

--接受数据并写入指定通讯类型变量通讯地址的数据

--socket ID 为 6
```

```
--通讯变量个数为 1000  
--通讯变量类型为 int16  
--通讯变量起始偏移量为 0  
--超时时间 1000ms
```

三 通讯功能的实现

3.1 机器人作为客户端

机器人作为客户端时，需要先确定服务器的通讯协议类型、IP 地址、socket ID 和端口号，确定完毕后按照 **2.3.1 机器人作为客户端** 格式写好程序，运行即可进行通讯。

3.2 机器人作为服务器

机器人作为服务器时，需要先定义服务器的通讯协议类型、IP 地址、socket ID 和端口号，然后将客户端的设置与服务器同步，按照 **2.3.2 机器人作为服务器** 格式写好程序，运行即可进行通讯。